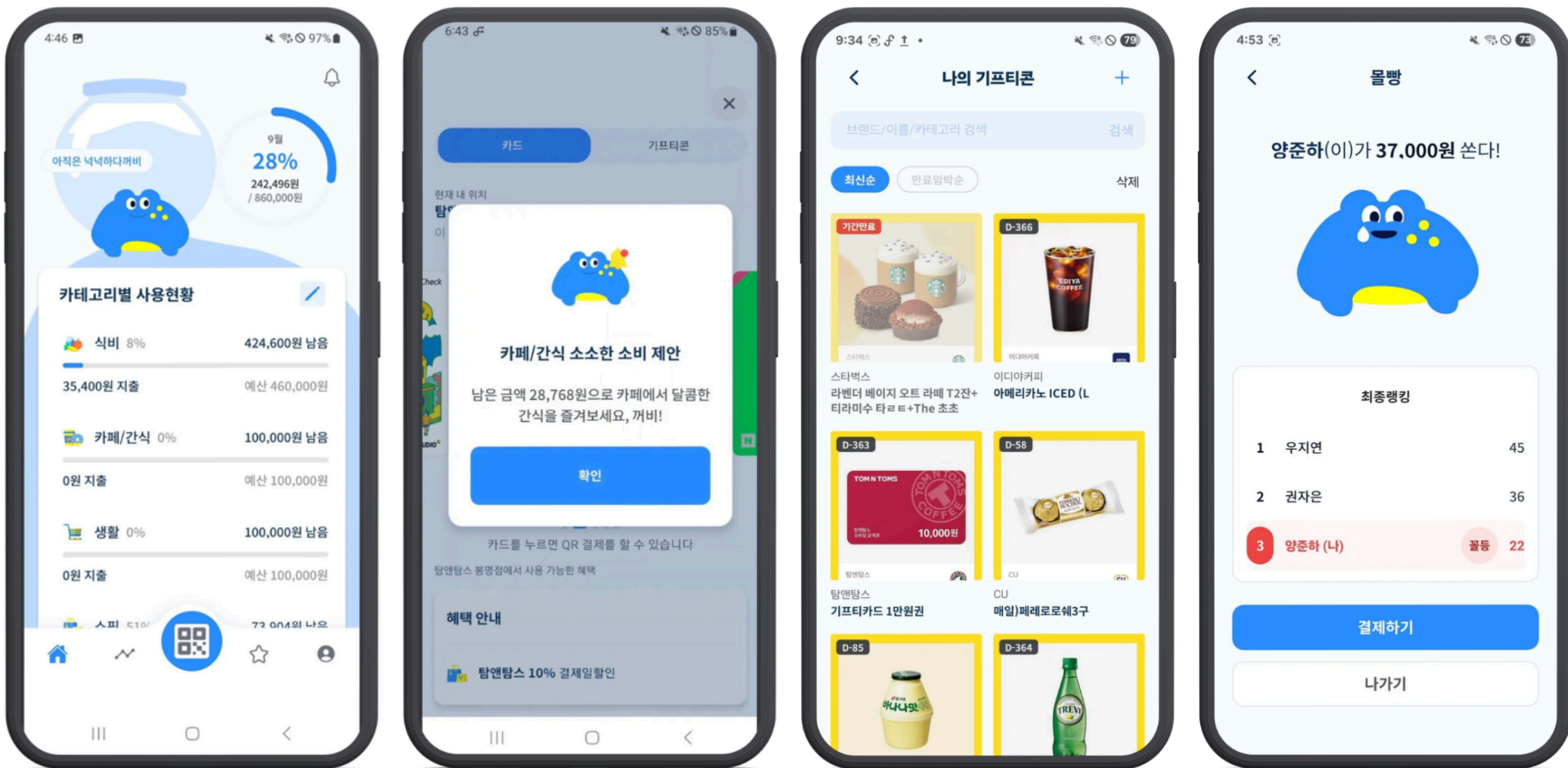




서비스

AI 기반 예산 관리·소비 코칭 모바일 앱 🏆 특화 PJT 우수상 1등

이상 소비 탐지·개인화 예산 추천·기프트콘 통합 관리·게이미피케이션으로 건전한 소비 습관 형성 지원

GitHub Repository

기술 스택



역할 — Backend

모바일 OAuth2 SSO

카카오 로그인 + 딥링크(toktokggubi://oauth/kakao) 복귀 + JWT RTR

OCR 파이프라인

Google Cloud Vision → 기프트콘 brand·label·expiresAt 파싱 → 정형 엔티티

실시간 동기화

N빵 계산기 / 물빵 대결 / 흔들기 점수 WebSocket 실시간 동기화

외부 API 통합

SSAFY 금융 OpenAPI / OAuth / OCR — 타임아웃·재시도·예외 격리

도메인 모델링

예산/소비/혜택/카드/기프트콘 핵심 엔티티 + 트랜잭션 설계

작업 기간

2025. 08. 25 - 2025. 09. 26 (5주)

SSAFY 13기 특화 PJT (핀테크 도메인)

구성원

총 6명

FE 3명 (RN/Expo) · BE 2명 (본인 포함) · Infra/AI 1명

프로젝트 회고

· 모바일 딥링크 OAuth2 SSO 직접 설계

- Authorization Code 교환 → 사용자 조회/생성 → JWT 발급 → 앱 딥링크 복귀
- 웹과 다른 모바일 앱 환경의 인증 흐름 끊김 없이 연결

· OCR → 도메인 파싱 파이프라인 구축

- Google Vision 결과(비정형) → 정형 엔티티(brand/label/expires) 변환·저장

· 이기종 외부 API 통합 안정화

- SSAFY 금융 / OAuth / OCR — 각기 다른 지연 시간을 타임아웃·재시도·격리로 흡수

· WebSocket 실시간 동기화에서 연결·재구독 이슈를 사용자 식별 방식으로 해결

PROBLEM → SOLUTION

"스타벅스 / Starbucks / 스타벅스 코리아"가 서로 다른 가맹점 ID로 저장되면 흔들기 후 매장 매칭 실패. Tesseract 한국어 OCR 정확도 한계
→ GCP Vision API + 도메인 텍스트 파서 2단 구조 + 캐노니컬 매핑으로 정형화.

Before vs After

BEFORE

자유 입력 + Tesseract 자체 OCR

- ✗ 브랜드 표기 흔들림 — "스타벅스 / Starbucks / 스타벅스 코리아" 다른 ID 저장
- ✗ 입력 부담 — 한 장당 4 칸 자유 입력 (12장이면 48칸)
- ✗ 만료일 형식 흔들림 — "2025.11.30 / -11-30 / 11월 30일"
- ✗ Tesseract 한국어 정확도 한계 — 폰트·해상도에 민감, 후처리 보강 필요
- ✗ 흔들기 매장 매칭 단계에서 본인 기프트콘 누락 발생

AFTER

GCP Vision API + GifticonTextParser

- ✓ OCR 정확도 — Google Cloud Vision API로 한글 인식을 안정화
- ✓ 도메인 분리 — OCR 결과를 GifticonTextParser에서 정형화
- ✓ 캐노니컬 매핑 — 브랜드 별칭 사전으로 동일 franchiseId 보장
- ✓ 정규식 LocalDate 파싱 — 모든 만료일 형식 통일 변환
- ✓ 이미지 한 장으로 brand·label·expiresAt 자동 등록

2단 분리 파이프라인 — 책임 분리로 정확도 + 유지보수성 확보



domain/gifticon/parser/GifticonTextParser.java

정규식 + 캐노니컬 매핑

```
@Component @RequiredArgsConstructor
public class GifticonTextParser {

    private final FranchiseCanonicalMap canonicalMap;

    // 만료일 정규식 - 다양한 형식 통합 파싱
    private static final Pattern EXPIRES_RE = Pattern.compile(
        "(\\Ww{4})[\\Ww\\.\\Ww-\\/\\n]\\Ww*(\\Ww{1,2})[\\Ww\\.\\Ww-\\/\\n]\\Ww*(\\Ww{1,2})"
    );

    public GifticonParseResult parse(String ocrText) {
        // 1. 브랜드 추출 + 캐노니컬 매핑
        String rawBrand = extractBrand(ocrText);
        Long franchiseId = canonicalMap.resolve(rawBrand);
        // "Starbucks" / "스타벅스" / "스벅" → 동일 ID

        // 2. 만료일 LocalDate 파싱
        LocalDate expiresAt = parseExpiresAt(ocrText);

        // 3. 상품명 - 브랜드 · 만료일 제외 후 추출
        String label = extractLabel(ocrText, rawBrand);

        return new GifticonParseResult(franchiseId, label, expiresAt);
    }

    private LocalDate parseExpiresAt(String text) {
        var m = EXPIRES_RE.matcher(text);
        if (!m.find()) return null;
        return LocalDate.of(
            Integer.parseInt(m.group(1)),
            Integer.parseInt(m.group(2)),
            Integer.parseInt(m.group(3))
        );
    }

    // 캐노니컬 매핑 사전
    public class FranchiseCanonicalMap {
        public Long resolve(String raw) {
            return switch (normalize(raw)) {
                case "스타벅스", "starbucks", "스벅" -> STARBUCKS_ID;
                case "이디야", "ediya" -> EDIYA_ID;
                default -> findBySimilarity(raw);
            };
        }
    }
}
```

자동 등록

사용자 입력 부담 ↓

이미지 1장 → brand·label·expiresAt 동시 추출

2단 구조

OCR / 도메인 분리

GCP Vision은 텍스트만, 정형화는 도메인 책임

동일 ID

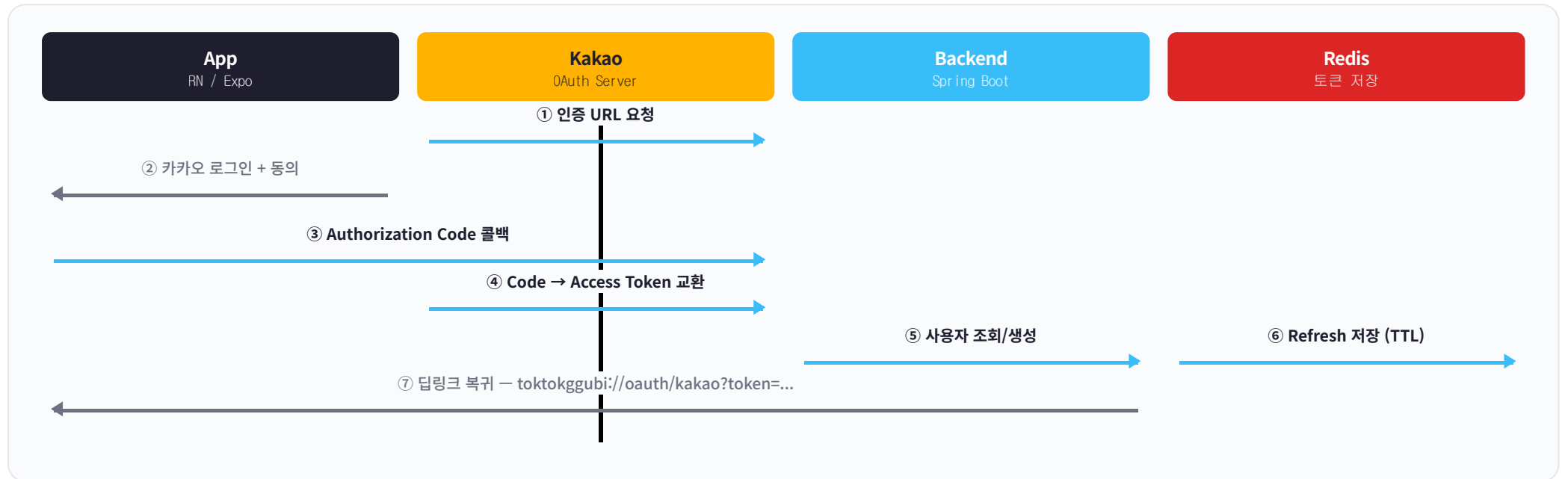
매장 매칭 정확도

캐노니컬 매핑으로 흔들기 결과 매칭 보장

PROBLEM → SOLUTION

웹 OAuth2와 달리 **앱 환경**은 브라우저 → 앱 복귀 흐름이 끊긴다. Authorization Code 교환 후 `toktokggubi://oauth/kakao` **커스텀 스킴 딥링크**로 안전 귀환 + JWT(Access/Refresh) 발급으로 이후 요청 인증.

OAuth2 + 딥링크 시퀀스



domain/auth/controller/AuthController.java

Kakao OAuth2 + 딥링크

```
@RestController @RequiredArgsConstructor
@RequestMapping("/auth/kakao")
public class AuthController {

    private final KakaoAuthService kakaoService;
    private final JwtProvider jwtProvider;

    // ① 앱이 호출 - 카카오 인증 URL 생성
    @GetMapping("/authorize-url")
    public AuthorizeUrlResponse authorizeUrl() {
        return new AuthorizeUrlResponse(kakaoService.buildAuthorizeUrl());
    }

    // ③~⑦ 카카오 콜백 - Code 교환 + 딥링크 복귀
    @GetMapping("/callback")
    public ResponseEntity<Void> callback(@RequestParam String code) {

        // 1. Authorization Code → Access Token 교환
        KakaoTokenResponse kakaoToken = kakaoService.exchangeToken(code);

        // 2. 카카오 프로필 조회 → 회원 조회/생성
        Member member = kakaoService.findOrCreate(kakaoToken);

        // 3. JWT 발급 (Access + Refresh, RTR)
        String accessToken = jwtProvider.issueAccess(member);
        String refreshToken = jwtProvider.issueRefresh(member);

        // 4. Refresh Token Redis 저장 (TTL)
        jwtProvider.saveRefresh(member.getId(), refreshToken);

        // 5. ★ 커스텀 스킴 딥링크로 복귀 - 앱이 자동 캐치
        String deepLink = "toktokggubi://oauth/kakao"
            + "?accessToken=" + accessToken
            + "&refreshToken=" + refreshToken;

        return ResponseEntity.status(HttpStatus.SEE_OTHER)
            .location(URI.create(deepLink))
            .build();
    }

    // Refresh Token Rotation - 탈취 시나리오 방어
    @PostMapping("/token/refresh")
    public TokenResponse refresh(@RequestBody RefreshRequest req) {
        return jwtProvider.rotate(req.refreshToken());
    }
}
```

APP RETURN

커스텀 스킴 딥링크

`toktokggubi://oauth/kakao` 로 앱이 등록한 스킴 사용. 외부 브라우저 → 앱 자동 복귀로 **인증 흐름 단절 없음**.

JWT FLOW

Access · Refresh 분리 + RTR

Access는 짧은 TTL · Refresh는 Redis TTL 보관. Refresh 쓰는 순간 **새 토큰 발급 + 이전 무효화**로 탈취 시나리오 방어.

FILTER + REVOCATION

JwtAuthenticationFilter

요청마다 JWT 검증 + `auth:blacklist:{jti}` 확인. `permitAll` 경로 정책으로 OAuth2 콜백 인증 우회.